WebAssembly :

De Javascript au C++ Haute Performance

Module : Développement Web Avancé
Cibles : Étudiants 2ème Année
Stack : AssemblyScript → C++ (Emscripten)

Roadmap du module

- **S1-S2 :** Découverte douce avec **AssemblyScript**.
- **S3 :** Le "Grand Saut" vers le **C++**.
- **S4-S5 :** Performance & Mémoire (Le cœur du sujet).
- **S6-S8 :** Écosystème & Projet Final.

Séance 1

Le Premier Contact

Pourquoi WebAssembly (Wasm) ?

Le JavaScript est génial, mais il a des limites pour le calcul intensif :

1. **Interprété / JIT :** Performance parfois imprévisible.
2. **Typage dynamique :** Coûteux en mémoire.
3. **Mono-thread :** Bloque l'UI si le calcul est trop lourd.

👉 **La solution Wasm :** Un format binaire, typé, exécuté à une vitesse proche du natif (C/C++/Rust).

Ce que Wasm N'EST PAS

- ❌ Un remplaçant du JavaScript (il vit *à côté*).
- ❌ Un langage de programmation (c'est une **cible** de compilation).
- ❌ Un moyen d'accéder au DOM directement (pour l'instant, on passe par JS).

✅ C'est un **coprocesseur** pour vos calculs lourds (Physique, Image, IA).

1. Qu'est-ce que le Bytecode ?

Comment le navigateur comprend votre code ?

1. **Code Source (Lourd) :** `function add(a,b)...` Trop lent à analyser en temps réel.
2. **Code Machine (Natif) :** `010110...` Dépend du processeur (PC vs Mac vs Mobile). Impossible sur le Web.
3. **Bytecode (Wasm) :** Un compromis.
   - Compact (téléchargement rapide).
   - Universel (indépendant du CPU).
   - Exécuté par une **Machine Virtuelle**.

> **Note :** C'est le même principe que Java ou PHP, mais standardisé pour le Web.

Exemple : La fonction `multiply` en Bytecode (Binaire)

Voici ce que voit réellement la machine virtuelle (en hexadécimal) :

```
00 61 73 6d 01 00 00 00
01 07 01 60 02 7f 7f 01
7f 03 02 01 00 07 0c 01
08 6d 75 6c 74 69 70 6c
79 00 00 0a 09 01 07 00
20 00 20 01 6c 0b
```

Ce format est conçu pour être **parcouru très rapidement** par le navigateur.

Spécification officielle : https://webassembly.github.io/spec/core/

La "Glue" JavaScript

Le navigateur ne comprend pas Wasm nativement comme une balise `<script>`. Il faut le charger.

```
// loader.js
const wasmUrl = "my_module.wasm";

// 1. Télécharger et instancier
const { instance } = await WebAssembly.instantiateStreaming(
    fetch(wasmUrl)
);

// 2. Utiliser les fonctions exportées
const { myFunction } = instance.exports;
console.log(myFunction(10, 20));
```

2. Sous le capot : Le format WAT

Wasm est un format binaire (`.wasm`), illisible pour un humain.
Mais il a une représentation textuelle : le **WAT** (.wat).

C'est une **Machine à Pile** (Stack Machine).

1. On empile des données.
2. Une opération (opcode) mange les données du haut de la pile.
3. Elle empile le résultat.

Exemple : Une addition en WAT

Pas de variables complexes ici, juste de la logique pure.

```
(module
  (func $add (export "add") (param $a i32) (param $b i32) (result i32)
    local.get $a    ;; PUSH paramètre A sur la pile
    local.get $b    ;; PUSH paramètre B sur la pile
    i32.add         ;; POP A, POP B, ADD, PUSH Result
  )
)
```

```
$ wat-wasm multiply.wat -o multiply.wasm
```

C'est verbeux ? Oui. C'est pour ça qu'on ne code pas en WAT, mais en **AssemblyScript**.

Exercice WAT : Carre (square)

En vous basant sur l'exemple précédent, comment écririez-vous la fonction `power` qui multiplie un nombre par lui-même ?

```
(func $power (export "square") (param $n i32) (result i32)
  ;; Votre code ici...
)
```

Challenge WAT : Power (Boucle)

Pour les courageux : implémentez l'élévation à la puissance.
Indices : `local.set`, `local.get`, `br_if` ou `loop`.

```
(func $pow (export "pow") (param $base i32) (param $exp i32) (result i32)
  ;; Structure suggérée :
  ;; 1. Variable locale pour le résultat (initialisée à 1)
  ;; 2. Boucle pour multiplier le résultat par $base, $exp fois
)
```

3. Notre outil de départ : AssemblyScript

Pourquoi ne pas commencer par le C++ tout de suite ?

- Syntaxe quasi-identique à **Javascript**.
- Typage strict (`i32` , `f64`) comme en C.
- Compilation immédiate via npm.

AssemblyScript vs JavaScript

Vous connaissez le JS. AssemblyScript y ressemble, mais c'est un piège !

AssemblyScript, c'est du C++ déguisé en JS.

JavaScript ⚡	AssemblyScript 🛡️
Typage Dynamique `let x = 10;` `x = "texte";` // OK	**Typage Statique Strict** `let x: i32 = 10;` `x = "texte";` // 💥 Erreur
Nombres Flous Tout est `number`	**Nombres Précis** `i32` (entier), `f64` (flottant), `u8` (octet)
Mémoire Auto Garbage Collector	**Mémoire Manuelle** Comme en C (Allocations)

Exemple : Hello World en AssemblyScript

Regardez les `: i32`. C'est la seule différence visible, mais elle change tout pour le processeur.

```
// index.ts

// Fonction qui prend deux ENTIERS 32 bits et renvoie un ENTIER 32 bits
export function add(a: i32, b: i32): i32 {
  return a + b;
}
```

```
$ asc add.ts -o add.wasm
```

Une fois compilé, cela devient exactement le code WAT vu précédemment !

Exercice AssemblyScript : Power

Écrivez une fonction `pow` qui prend une **base** et un **exposant** (entiers 32 bits) et retourne le résultat.

Indice : Utilisez une boucle `for` ou la récursion.

```
export function pow(base: i32, exp: i32): i32 {
  // Votre code ici...
}
```

Challenge AssemblyScript : Fibonacci

Implémentez une fonction qui calcule le n-ième nombre de la suite de Fibonacci.

- **Contrainte :** Utilisez le type `u64` pour le résultat afin de gérer des nombres très grands.
- **Objectif :** Comparer la vitesse d'exécution entre la version récursive (lente) et itérative (rapide).

```
export function fibo(n: i32): u64 {
  // Votre code ici...
}
```